scikit-learn Cross-validation & Classification

Lecture 13

Dr. Colin Rundel

Cross validation & hyperparameter tuning

Ridge regression

One way to expand on the idea of least squares regression is to modify the loss function. Ridge regression is one such approach - it adds a scaled penalty for the sum of the squared βs to the least squares loss.

$$\underset{\boldsymbol{\beta}}{\operatorname{argmin}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 + \lambda(\boldsymbol{\beta}^T\boldsymbol{\beta})$$

```python
d = pd.read_csv("data/ridge.csv"); d
```

```
            y        x1        x2        x3        x4 x5
0   -0.151710  0.353658  1.633932  0.553257  1.415731  A
1    3.579895  1.311354  1.457500  0.072879  0.330330  B
2    0.768329 -0.744034  0.710362 -0.246941  0.008825  B
3    7.788646  0.806624 -0.228695  0.408348 -2.481624  B
4    1.394327  0.837430 -1.091535 -0.860979 -0.810492  A
..        ...       ...       ...       ...       ... ..
495 -0.204932 -0.385814 -0.130371 -0.046242  0.004914  A
496  0.541988  0.845885  0.045291  0.171596  0.332869  A
497 -1.402627 -1.071672 -1.716487 -0.319496 -1.163740  C
498 -0.043645  1.744800 -0.010161  0.422594  0.772606  A
499 -1.550276  0.910775 -1.675396  1.921238 -0.232189  B

[500 rows x 6 columns]
```

dummy coding

```python
d = pd.get_dummies(d); d
```

```
            y        x1        x2        x3  ...   x5_A   x5_B   x5_C   x5_D
0   -0.151710  0.353658  1.633932  0.553257  ...   True  False  False  False
1    3.579895  1.311354  1.457500  0.072879  ...  False   True  False  False
2    0.768329 -0.744034  0.710362 -0.246941  ...  False   True  False  False
3    7.788646  0.806624 -0.228695  0.408348  ...  False   True  False  False
4    1.394327  0.837430 -1.091535 -0.860979  ...   True  False  False  False
..        ...       ...       ...       ...  ...    ...    ...    ...    ...
495 -0.204932 -0.385814 -0.130371 -0.046242  ...   True  False  False  False
496  0.541988  0.845885  0.045291  0.171596  ...   True  False  False  False
497 -1.402627 -1.071672 -1.716487 -0.319496  ...  False  False   True  False
498 -0.043645  1.744800 -0.010161  0.422594  ...   True  False  False  False
499 -1.550276  0.910775 -1.675396  1.921238  ...  False   True  False  False

[500 rows x 9 columns]
```

Fitting a ridge regression model

The `linear_model` submodule also contains the `Ridge` model which can be used to fit a ridge regression model. Usage is identical other than `Ridge()` takes the parameter `alpha` to specify the regularization parameter.

```
from sklearn.linear_model import Ridge, LinearRegression

X, y = d.drop(["y"], axis=1), d.y

lm = LinearRegression(fit_intercept=False).fit(X, y)
rg = Ridge(fit_intercept=False, alpha=10).fit(X, y)
```

```
lm.coef_
```

```
array([ 0.99505,  2.00762,  0.00232, -3.00088,
```

```
root_mean_squared_error(y, lm.predict(X))
```

```
0.0993601324682191
```

```
rg.coef_
```

```
array([ 0.97809,  1.96215,  0.00172, -2.94457,
```

```
root_mean_squared_error(y, rg.predict(X))
```

```
0.13820792795597317
```

Generally for a Ridge (or Lasso) model it is important to scale the features before fitting (i.e. `StandardScaler()`) - in this

Test-Train split

The most basic form of CV is to split the data into a testing and training set, this can be achieved using `train_test_split` from the `model_selection` submodule.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=1234
)
```

```
X.shape
```

```
(500, 8)
```

```
y.shape
```

```
(500,)
```

```
X_train.shape
```

```
(400, 8)
```

```
y_train.shape
```

```
(400,)
```

```
X_test.shape
```

```
(100, 8)
```

```
y_test.shape
```

```
(100,)
```

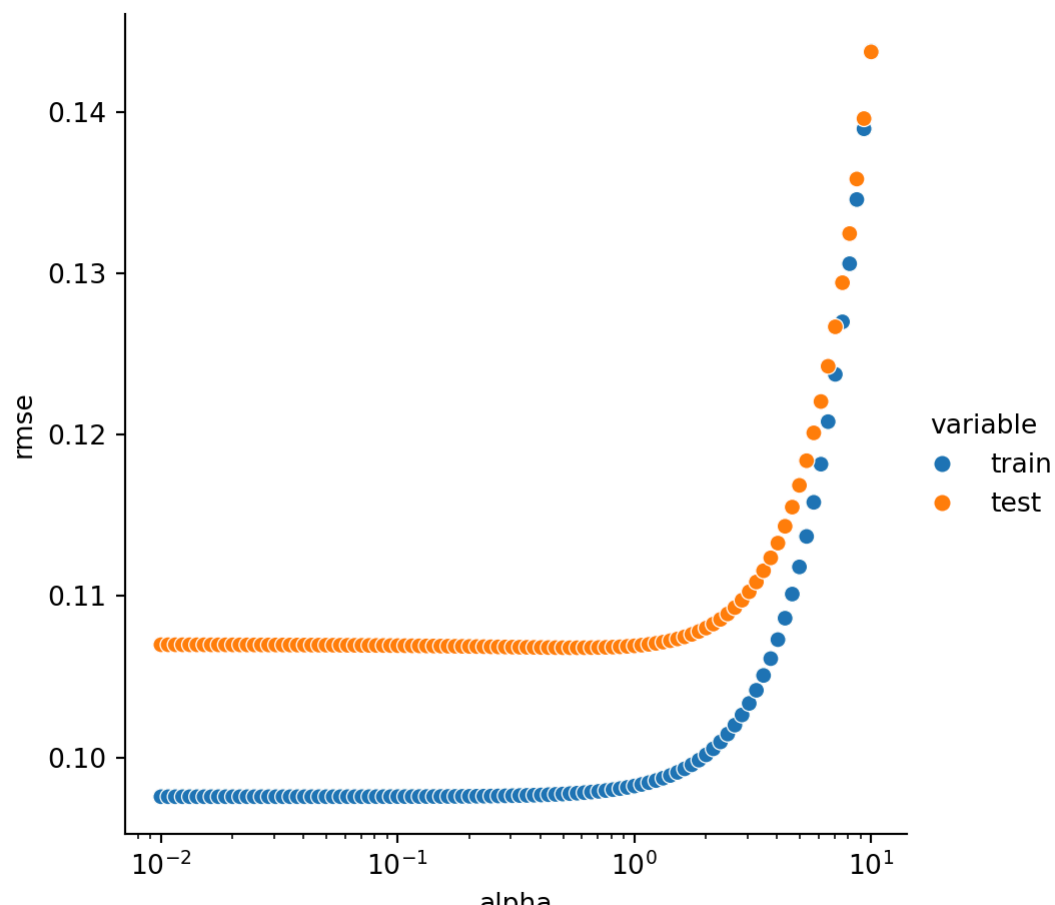
Train vs Test rmse

```python
alpha = np.logspace(-2,1, 100)
train_rmse = []
test_rmse = []

for a in alpha:
    rg = Ridge(alpha=a).fit(
      X_train, y_train
    )
    train_rmse.append(
     root_mean_squared_error(
        y_train, rg.predict(X_train)
      )
    )
    test_rmse.append(
      root_mean_squared_error(
        y_test, rg.predict(X_test)
      )
    )

res = pd.DataFrame( {
  "alpha": alpha,
   "train": train_rmse,
   "test": test_rmse
```

```python
res
```

```
        alpha     train      test
0    0.010000  0.097568  0.106985
1    0.010723  0.097568  0.106984
2    0.011498  0.097568  0.106984
3    0.012328  0.097568  0.106983
4    0.013219  0.097568  0.106983
..        ...       ...       ...
95   7.564633  0.126990  0.129414
96   8.111308  0.130591  0.132458
97   8.697490  0.134568  0.135838
98   9.326033  0.138950  0.139581
99  10.000000  0.143764  0.143715

[100 rows x 3 columns]
```

```
g = sns.relplot(
  x="alpha", y="rmse", hue="variable", data = pd.melt(res, id_vars=["alpha"],value_name="rmse
).set(
  xscale="log"
)
```



Best alpha?

```
min_i = np.argmin(res.train)
min_i
```

```
np.int64(0)
```

```
res.iloc[[min_i],:]
```

```
   alpha     train      test
0   0.01  0.097568  0.106985
```

```
min_i = np.argmin(res.test)
min_i
```

```
np.int64(58)
```

```
res.iloc[[min_i],:]
```

```
       alpha     train    test
58  0.572237  0.097787  0.1068
```

k-fold cross validation

The previous approach was relatively straightforward, but it required a fair bit of bookkeeping to implement and we only examined a single test/train split. If we would like to perform k-fold cross validation we can use `cross_val_score` from the `model_selection` submodule.

```
from sklearn.model_selection import cross_val_score

cross_val_score(
  Ridge(alpha=0.59, fit_intercept=False),
  X, y,
  cv=5,
  scoring="neg_root_mean_squared_error"
)
```

```
array([-0.09364, -0.09995, -0.10474, -0.10273, -0.10597])
```

🚩🚩🚩 Note that the default k-fold cross validation used here does not shuffle the data which can be massively

Controlling k-fold behavior

Rather than providing `cv` as an integer, it is better to specify a cross-validation scheme directly (with additional options). Here we will use the `KFold` class from the `model_selection` submodule.

```
from sklearn.model_selection import KFold

cross_val_score(
  Ridge(alpha=0.59, fit_intercept=False),
  X, y,
  cv = KFold(n_splits=5, shuffle=True, random_state=1234),
  scoring="neg_root_mean_squared_error"
)
```

```
array([-0.10658, -0.104  , -0.1037 , -0.10125, -0.09228])
```

KFold object

`KFold()` returns a class object which provides the method `split()` which in turn is a generator that returns a tuple with the indexes of the training and test selections for each fold given a model matrix `X`,

```
ex = pd.DataFrame(data = list(range(10)), columns=["x"])
```

```
cv = KFold(5)
for train, test in cv.split(ex):
  print(f'Train: {train} | test: {test}')
```

```
Train: [2 3 4 5 6 7 8 9] | test: [0 1]
Train: [0 1 4 5 6 7 8 9] | test: [2 3]
Train: [0 1 2 3 6 7 8 9] | test: [4 5]
Train: [0 1 2 3 4 5 8 9] | test: [6 7]
Train: [0 1 2 3 4 5 6 7] | test: [8 9]
```

```
cv = KFold(5, shuffle=True, random_state=123
for train, test in cv.split(ex):
  print(f'Train: {train} | test: {test}')
```

```
Train: [0 1 3 4 5 6 8 9] | test: [2 7]
Train: [0 2 3 4 5 6 7 8] | test: [1 9]
Train: [1 2 3 4 5 6 7 9] | test: [0 8]
Train: [0 1 2 3 6 7 8 9] | test: [4 5]
Train: [0 1 2 4 5 7 8 9] | test: [3 6]
```

scoring

For most of the cross validation functions we pass in a string instead of a scoring function from the metrics submodule - if you are interested in seeing the names of the possible metrics, these are available via `sklearn.metrics.get_scorer_names()`,

```
np.array( sklearn.metrics.get_scorer_names() )
```

```
array(['accuracy', 'adjusted_mutual_info_score', 'adjusted_rand_score',
       'average_precision', 'balanced_accuracy', 'completeness_score',
       'd2_absolute_error_score', 'd2_brier_score', 'd2_log_loss_score',
       'explained_variance', 'f1', 'f1_macro', 'f1_micro', 'f1_samples', 'f1_weighted',
       'fowlkes_mallows_score', 'homogeneity_score', 'jaccard', 'jaccard_macro',
       'jaccard_micro', 'jaccard_samples', 'jaccard_weighted', 'matthews_corrcoef',
       'mutual_info_score', 'neg_brier_score', 'neg_log_loss', 'neg_max_error',
       'neg_mean_absolute_error', 'neg_mean_absolute_percentage_error',
       'neg_mean_gamma_deviance', 'neg_mean_poisson_deviance', 'neg_mean_squared_error',
       'neg_mean_squared_log_error', 'neg_median_absolute_error',
       'neg_negative_likelihood_ratio', 'neg_root_mean_squared_error',
       'neg_root_mean_squared_log_error', 'normalized_mutual_info_score',
       'positive_likelihood_ratio', 'precision', 'precision_macro', 'precision_micro',
       'precision_samples', 'precision_weighted', 'r2', 'rand_score', 'recall',
       'recall_macro', 'recall_micro', 'recall_samples', 'recall_weighted', 'roc_auc',
       'roc_auc_ovo', 'roc_auc_ovo_weighted', 'roc_auc_ovr', 'roc_auc_ovr_weighted',
       'top_k_accuracy', 'v_measure_score'], dtype='<U34')
```

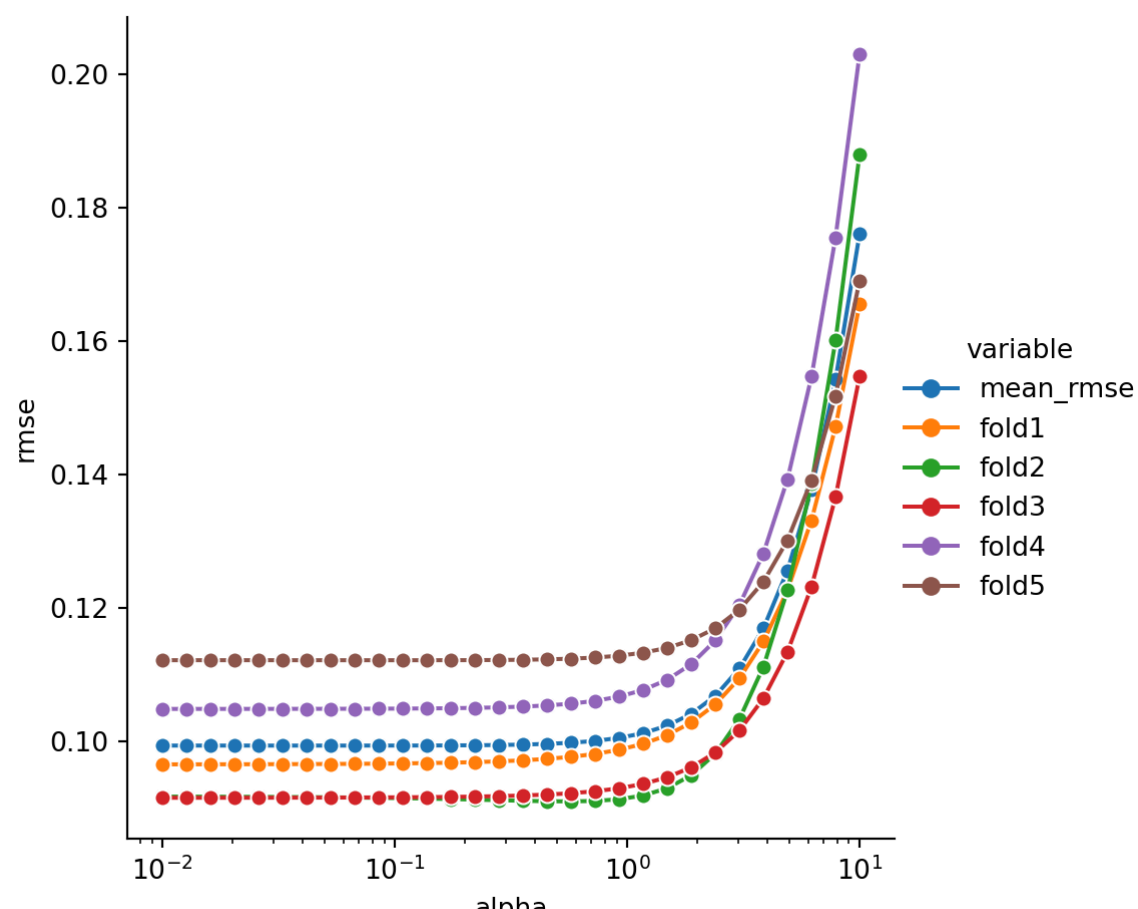
Train vs Test rmse (again)

```
alpha = np.logspace(-2,1, 30)
test_mean_rmse = []
test_rmse = []
cv = KFold(n_splits=5, shuffle=True, random_state=1234)

for a in alpha:
    rg = Ridge(fit_intercept=False, alpha=a)

    scores = -1 * cross_val_score(
      rg, X_train, y_train,
      cv = cv,
      scoring="neg_root_mean_squared_error"
    )
    test_mean_rmse.append(np.mean(scores))
    test_rmse.append(scores)

res = pd.DataFrame(
    data = np.c_[alpha, test_mean_rmse, test_rmse],
    columns = ["alpha", "mean_rmse"] + ["fold" + str(i) for i in range(1,6) ]
)
```

```r
res
```

	alpha	mean_rmse	fold1	fold2	fold3	fold4	fold5
0	0.010000	0.099393	0.096577	0.091750	0.091573	0.104881	0.112186
1	0.012690	0.099393	0.096581	0.091743	0.091575	0.104882	0.112185
2	0.016103	0.099393	0.096585	0.091734	0.091577	0.104884	0.112185
3	0.020434	0.099392	0.096591	0.091722	0.091580	0.104885	0.112184
4	0.025929	0.099392	0.096599	0.091708	0.091583	0.104888	0.112183
5	0.032903	0.099392	0.096608	0.091690	0.091588	0.104891	0.112182
6	0.041753	0.099391	0.096621	0.091667	0.091594	0.104895	0.112181
7	0.052983	0.099392	0.096637	0.091639	0.091602	0.104900	0.112180
8	0.067234	0.099392	0.096657	0.091604	0.091612	0.104908	0.112179
9	0.085317	0.099394	0.096684	0.091561	0.091626	0.104919	0.112179
10	0.108264	0.099398	0.096720	0.091510	0.091644	0.104935	0.112179
11	0.137382	0.099405	0.096767	0.091448	0.091669	0.104958	0.112181
12	0.174333	0.099417	0.096829	0.091376	0.091704	0.104992	0.112186
13	0.221222	0.099439	0.096913	0.091294	0.091751	0.105042	0.112196
14	0.280722	0.099477	0.097028	0.091207	0.091819	0.105117	0.112215
15	0.356225	0.099540	0.097185	0.091121	0.091914	0.105231	0.112249
16	0.452035	0.099644	0.097403	0.091052	0.092052	0.105406	0.112309
17	0.573615	0.099816	0.097709	0.091030	0.092254	0.105675	0.112410
18	0.727895	0.100093	0.098143	0.091102	0.092551	0.106090	0.112580
19	0.923671	0.100541	0.098766	0.091354	0.092993	0.106733	0.112857
20	1.172102	0.101254	0.099664	0.091918	0.093654	0.107727	0.113309

```
g = sns.relplot(
  x="alpha", y="rmse", hue="variable", data=res.melt(id_vars=["alpha"], value_name
  marker="o", kind="line"
).set(
  xscale="log"
)
```



Grid Search

We can further reduce the amount of code needed if there is a specific set of parameter values we would like to explore using cross validation. This is done using the `GridSearchCV` function from the `model_selection` submodule.

```
from sklearn.model_selection import GridSearchCV

gs = GridSearchCV(
  Ridge(fit_intercept=False),
  param_grid = {"alpha": np.logspace(-2, 1, 30)},
  cv = KFold(5, shuffle=True, random_state=1234),
  scoring = "neg_root_mean_squared_error"
).fit(
  X, y
)
```

`best_*` attributes

`GridSearchCV()`'s return object contains attributes with details on the "best" model based on the chosen scoring metric.

```
gs.best_index_
```

```
np.int64(5)
```

```
gs.best_params_
```

```
{'alpha': np.float64(0.03290344562312668)}
```

```
gs.best_score_
```

```
np.float64(-0.1012561176745365)
```

`best_estimator_` attribute

If `refit = True` (default) with `GridSearchCV()` then the `best_estimator_` attribute will be available which gives direct access to the "best" model or pipeline object. This model is constructed by using the parameter(s) that achieved the minimum score and refitting the model to the complete data set.

```
gs.best_estimator_
```

```
Ridge(alpha=np.float64(0.03290344562312668), fit_intercept=False)
```

```
gs.best_estimator_.coef_
```

```
array([ 0.99499,  2.00747,  0.00231, -3.0007 ,  0.49316,  0.10189, -0.29408,  1.00767])
```

```
gs.best_estimator_.predict(X)
```

```
array([ -0.12179,   3.34151,   0.76055,   7.89292,   1.56523,  -5.33575,  -4.37469,
         3.13003,  -0.16859,  -1.60087,  -1.89073,   1.44596,   3.99773,   4.70003,
        -6.45959,   4.11085,   3.60426,  -1.96548,   2.99039,   0.56796,  -5.26672,
         5.4966 ,   3.47247,  -2.66117,   3.35011,   0.64221,  -1.50238,   2.41562,
         3.11665,   1.11236,  -2.11839,   1.36006,  -0.53666,  -2.78112,   0.76008,
         5.49779,   2.6521 ,  -0.83127,   0.04167,  -1.92585,  -2.48865,   2.29127,
         3.62514,  -2.01226,  -0.69725,  -1.94514,  -0.47559,  -7.36557,  -3.20766,
         2.9218 ,  -0.8213 ,  -2.78598, -12.55143,   2.79189,  -1.89763,  -5.1769 ,
         1.87484,   2.18345,  -6.45358,   0.91006,   0.94792,   2.91799,   6.12323,
        -1.87654,   3.63259,  -0.53797,  -3.23506,  -2.23885,   1.04564,  -1.54843,
         0.76161,  -1.65495,   0.22378,  -0.68221,   0.12976,   2.58875,   2.54421,
        -3.69056,   3.73479,  -0.90278,   1.22394,  -3.22614,   7.16719,  -5.6168 ,
```

```
 3.3433 ,   0.36935,   0.87397,   9.22348,  -1.29078,   1.74347,  -1.55169,
-0.69398,  -1.40445,   0.23072,   1.06277,   2.84797,   2.35596,  -1.93292,
 8.35129,  -2.98221,  -6.35071,  -5.15138,   1.70208,   7.15821,   3.96172,
 5.75363,  -4.50718,  -5.81785,  -2.47424,   1.19276,   2.57431,  -2.57053,
-0.53682,  -1.65955,   1.99839,  -6.19607,  -1.73962,  -2.11993,  -2.29362,
 2.65413,  -0.67486,  -3.01324,   0.34118,  -3.83856,   0.33096,  -3.59485,
-1.55578,   0.96765,   3.50934,  -0.31935,  -4.18323,   2.87843,  -1.64857,
-3.68181,   2.24423,  -1.00244,  -2.65588,  -5.77111,  -1.20292,   2.66903,
-1.11387,   3.05231,   6.34596,  -1.42886,  -2.29709,  -1.4573 ,  -2.46733,
 1.69685,   4.21699,   1.21569,   9.06269,  -3.62209,   1.94704,   1.14603,
```

cv_results_ attribute

Other useful details about the grid search process are stored as a dictionary in the `cv_results_` attribute which includes things like average test scores, fold level test scores, test ranks, test runtimes, etc.

```
gs.cv_results_.keys()
```

```
dict_keys(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time', 'param_alpha', 'p
```

```
gs.cv_results_["mean_test_score"]
```

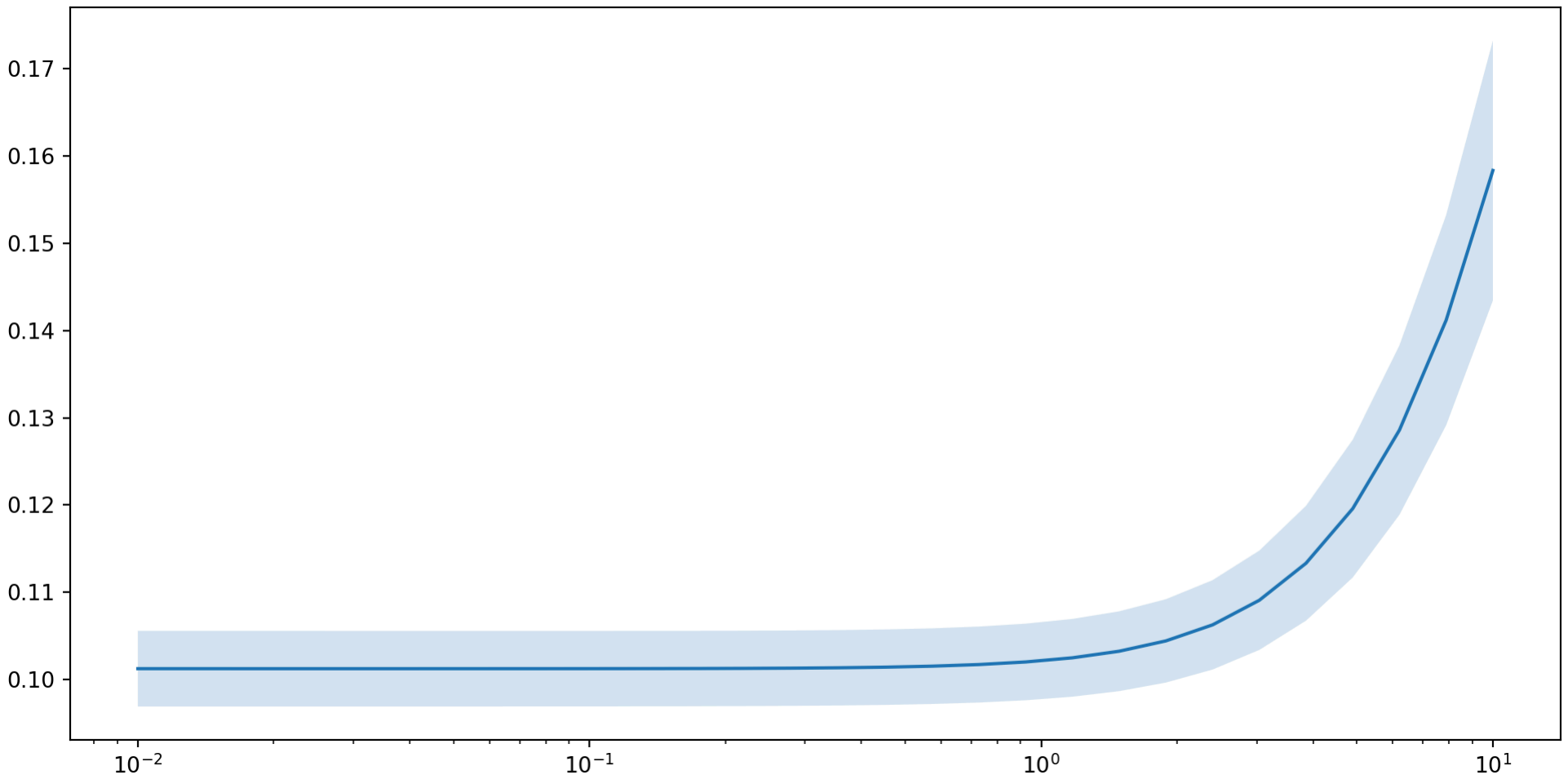
```
array([-0.10126, -0.10126, -0.10126, -0.10126, -0.10126, -0.10126, -0.10126, -0.10126,
       -0.10126, -0.10126, -0.10127, -0.10128, -0.10129, -0.10132, -0.10136,
       -0.10143, -0.10154, -0.10173, -0.10203, -0.1025 , -0.10325, -0.10444, -0.10627,
       -0.10909, -0.11333, -0.11959, -0.12859, -0.14119, -0.15832])
```

```
gs.cv_results_["param_alpha"]
```

```
masked_array(data=[0.01, 0.01268961003167922, 0.01610262027560939, 0.020433597178569417,
                   0.02592943797404667, 0.03290344562312668, 0.041753189365604,
                   0.05298316906283707, 0.06723357536499334, 0.08531678524172806,
                   0.10826367338740546, 0.1373823795883263, 0.17433288221999882,
                   0.2212216291070449, 0.2807216203941177, 0.3562247890262442,
                   0.4520353656360243, 0.5736152510448679, 0.727895384398315,
                   0.9236708571873861, 1.1721022975334805, 1.4873521072935119,
                   1.8873918221350976, 2.395026619987486, 3.039195382313198,
                   3.856620421163472, 4.893900918477494, 6.210169418915616 5,
                   7.880462815669913, 10.0],
```

```
      mask=[False, False, False, False, False, False, False, False, False, False,
            False, False, False, False, False, False, False, False, False, False,
            False, False, False, False, False, False, False, False, False, False],
fill_value=1e+20)
```

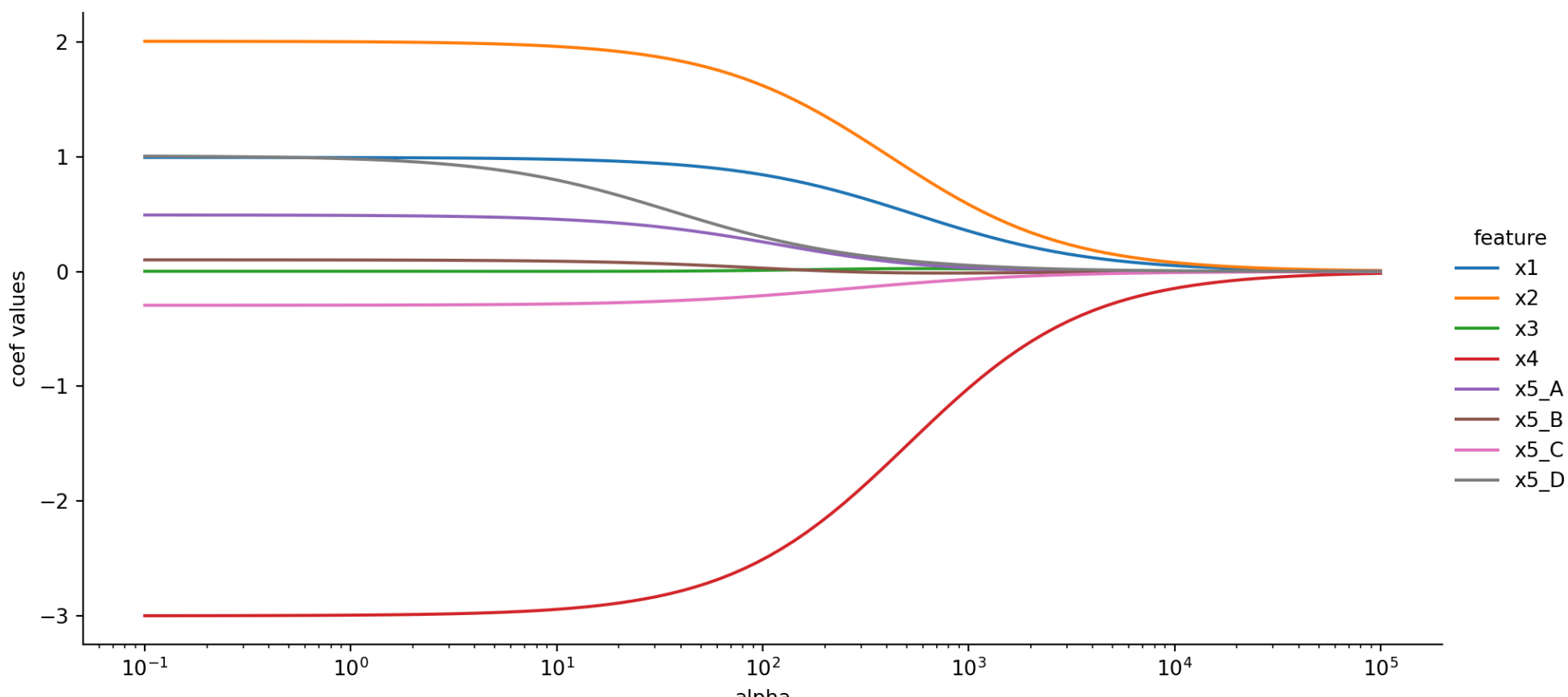
```python
alpha = np.array(gs.cv_results_["param_alpha"], dtype="float64")
score = -gs.cv_results_["mean_test_score"]
score_std = gs.cv_results_["std_test_score"]
n_folds = gs.cv.get_n_splits()

plt.figure(layout="constrained")
ax = sns.lineplot(x=alpha, y=score)
ax.set_xscale("log")
plt.fill_between(
  x = alpha,
  y1 = score + 1.96*score_std / np.sqrt(n_folds),
  y2 = score - 1.96*score_std / np.sqrt(n_folds),
  alpha = 0.2
)
plt.show()
```


0.17
0.16
0.15
0.14
0.13
0.12
0.11
0.10
10^{-2}
10^{-1}
10^{0}
10^{1}

Ridge traceplot

```
alpha = np.logspace(-1,5, 100)
betas = []

for a in alpha:
    rg = Ridge(alpha=a, fit_intercept=False).fit(X, y)
    betas.append(rg.coef_)

res = pd.DataFrame(
  data = betas, columns = rg.feature_names_in_
).assign(
  alpha = alpha
)
```

```python
g = sns.relplot(
  data = res.melt(id_vars="alpha", value_name="coef values", var_name="feature"),
  x = "alpha", y = "coef values", hue = "feature",
  kind = "line", aspect=2
).set(
  xscale="log"
)
```



Classification

OpenIntro - Spam

We will start by looking at a data set on spam emails from the OpenIntro project. A full data dictionary can be found here. To keep things simple this week we will restrict our exploration to including only the following columns: `spam`, `exclaim_mess`, `format`, `num_char`, `line_breaks`, and `number`.

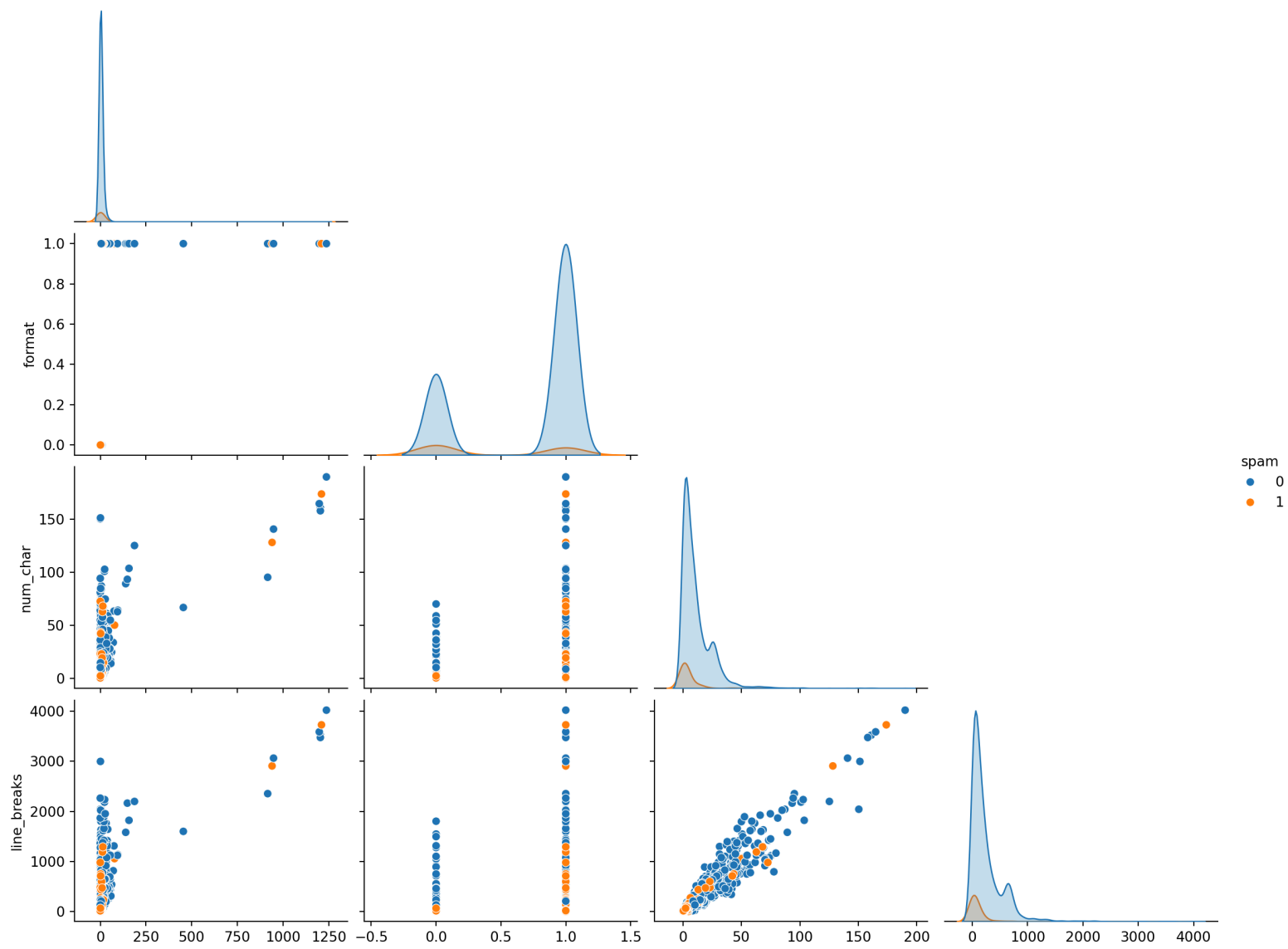
- `spam` - Indicator for whether the email was spam.
- `exclaim_mess` - The number of exclamation points in the email message.
- `format` - Indicates whether the email was written using HTML (e.g. may have included bolding or active links).
- `num_char` - The number of characters in the email, in thousands.
- `line_breaks` - The number of line breaks in the email (does not count text wrapping).
- `number` - Factor variable saying whether there was no number, a small number (under 1 million), or a big number.

As `number` is categorical, we will take care of the necessary dummy coding via `pd.get_dummies()`,

```
email = pd.read_csv('data/email.csv')[
  ['spam', 'exclaim_mess', 'format', 'num_char', 'line_breaks', 'number']
]
email_dc = pd.get_dummies(email)
email_dc
```

```
      spam  exclaim_mess  format  ...  number_big  number_none  number_small
0        0             0       1  ...        True        False         False
1        0             1       1  ...       False        False          True
2        0             6       1  ...       False        False          True
3        0            48       1  ...       False        False          True
4        0             1       0  ...       False         True         False
...    ...           ...     ...  ...         ...          ...           ...
3916     1             0       0  ...       False        False          True
3917     1             0       0  ...       False        False          True
3918     0             5       1  ...       False        False          True
3919     0             0       0  ...       False        False          True
3920     1             1       0  ...       False        False          True

[3921 rows x 8 columns]
```

```python
g = sns.pairplot(email, hue='spam', corner=True, aspect=1.25)
```



Model fitting

```python
from sklearn.linear_model import LogisticRegression

y = email_dc.spam
X = email_dc.drop('spam', axis=1)

m = LogisticRegression(fit_intercept = False).fit(X, y)
```

```python
m.feature_names_in_
```

```
array(['exclaim_mess', 'format', 'num_char', 'line_breaks', 'number_big', 'number_none',
       'number_small'], dtype=object)
```

```python
m.coef_
```

```
array([[ 0.00982, -0.61905,  0.05449, -0.00555, -1.21236, -0.69326, -1.92064]])
```

A quick comparison

R output

```r
glm(spam~.-1, data=d, family=binomial) |>
  coef()
```

```
exclaim_mess       format     num_char  line_breaks    numberbig   numbernone
 0.009586821 -0.604781649  0.054765496 -0.005480427 -1.264826746 -0.706842516
 numbersmall
-1.950440237
```

sklearn output

```python
m.feature_names_in_
```

```
array(['exclaim_mess', 'format', 'num_char', 'line_breaks', 'number_big', 'number_none',
       'number_small'], dtype=object)
```

```python
m.coef_
```

```
array([[ 0.00982, -0.61905,  0.05449, -0.00555, -1.21236, -0.69326, -1.92064]])
```

`sklearn.linear_model.LogisticRegression`

From the documentation,

> This class implements regularized logistic regression using the ‘liblinear’ library, ‘newton-cg’, ‘sag’, ‘saga’ and ‘lbfgs’ solvers. **Note that regularization is applied by default.** It can handle both dense and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied).

Penalty parameter

🚩🚩🚩

`LogisticRegression()` has a parameter called penalty that applies a `"l1"` (lasso), `"l2"` (ridge), `"elasticnet"` or `None` with `"l2"` being the default. To make matters worse, the regularization is controlled by the parameter `C` which defaults to 1. `C` here is the inverse regularization strength (e.g. the inverse of `alpha` for ridge and lasso models).

🚩🚩🚩

$$\min_{w} \sum_{i=1}^{n} (-y_i \log(\hat{p}(X_i)) - (1 - y_i) \log(1 - \hat{p}(X_i))) + \frac{1}{C} \left(\frac{1 - \rho}{2} w^T w + \rho \|w\|_1 \right)$$

Another quick comparison

R output

```r
glm(spam~.-1, data = d, family=binomial) |>
  coef()
```

```
exclaim_mess       format     num_char  line_breaks    numberbig   numbernone
 0.009586821 -0.604781649  0.054765496 -0.005480427 -1.264826746 -0.706842516
 numbersmall
-1.950440237
```

sklearn output (penalty None)

```python
m = LogisticRegression(
  fit_intercept = False, penalty=None
).fit(X, y)
m.feature_names_in_
```

```
array(['exclaim_mess', 'format', 'num_char', 'line_breaks', 'number_big', 'number_none',
       'number_small'], dtype=object)
```

```python
m.coef_
```

```
array([[ 0.00959, -0.60483,  0.05476, -0.00548, -1.26481, -0.70687, -1.95043]])
```

Solver parameter

It is also possible to specify the solver to use when fitting a logistic regression model, to complicate matters somewhat the choice of the algorithm depends on the penalty chosen:

- `newton-cg` - `["l2",None]`
- `lbfgs` - `["l2",None]`
- `liblinear` - `["l1","l2"]`
- `sag` - `["l2",None]`
- `saga` - `["elasticnet","l1","l2",None]`

Also there can be issues with feature scales for some of these solvers:

> **Note:** 'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from sklearn.preprocessing.

Prediction

Classification models have multiple prediction methods depending on what type of output you would like,

```
m.predict(X)
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, ..., 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0], shape=(3921,))
```

```python
m.predict_proba(X)
```

```
array([[0.91325, 0.08675],
       [0.95595, 0.04405],
       [0.95788, 0.04212],
       [0.94085, 0.05915],
       [0.68757, 0.31243],
       [0.6845 , 0.3155 ],
       [0.93419, 0.06581],
       [0.96357, 0.03643],
       [0.89585, 0.10415],
       [0.94176, 0.05824],
       [0.93248, 0.06752],
       [0.89601, 0.10399],
       [0.91243, 0.08757],
       [0.97272, 0.02728],
       [0.92833, 0.07167],
       [0.98352, 0.01648],
       [0.96326, 0.03674],
       [0.95381, 0.04619],
       [0.88889, 0.11111],
       [0.80419, 0.19581],
       [0.89905, 0.10095],
       [0.95645, 0.04355],
```

```python
m.predict_log_proba(X)
```

```
array([[-0.09074, -2.44476],
       [-0.04505, -3.12251],
       [-0.04303, -3.16731],
       [-0.06098, -2.8276 ],
       [-0.37459, -1.16338],
       [-0.37906, -1.1536 ],
       [-0.06808, -2.72095],
       [-0.03711, -3.31224],
       [-0.10999, -2.26189],
       [-0.06   , -2.84318],
       [-0.0699 , -2.6954 ],
       [-0.10981, -2.26344],
       [-0.09165, -2.43527],
       [-0.02766, -3.60153],
       [-0.07436, -2.63572],
       [-0.01662, -4.10549],
       [-0.03743, -3.3039 ],
       [-0.04729, -3.07497],
       [-0.11778, -2.19724],
       [-0.21791, -1.63063],
       [-0.10642, -2.29309],
       [-0.04453, -3.13385],
```

Scoring

Classification models also include a `score()` method which returns the model's *accuracy*,

```
m.score(X, y)
```

```
0.90640142820709
```

Other scoring options are available via the metrics submodule

```
from sklearn.metrics import accuracy_score, roc_auc_score, f1_score, confusion_matrix
```

```
accuracy_score(y, m.predict(X))
```

```
0.90640142820709
```

```
roc_auc_score(y, m.predict_proba(X)[:,1])
```

```
0.7606967779329887
```

```
f1_score(y, m.predict(X))
```

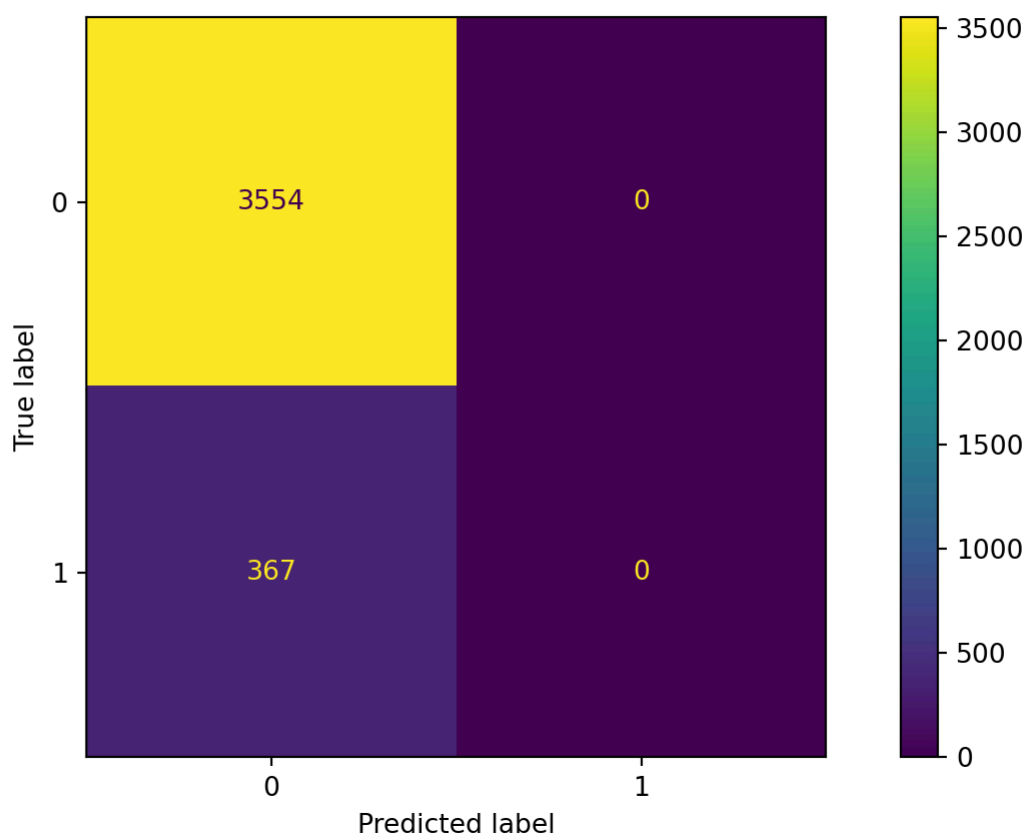
```
0.0
```

```
confusion_matrix(y, m.predict(X), labels=m.
```

```
array([[3554,    0],
       [ 367,    0]])
```

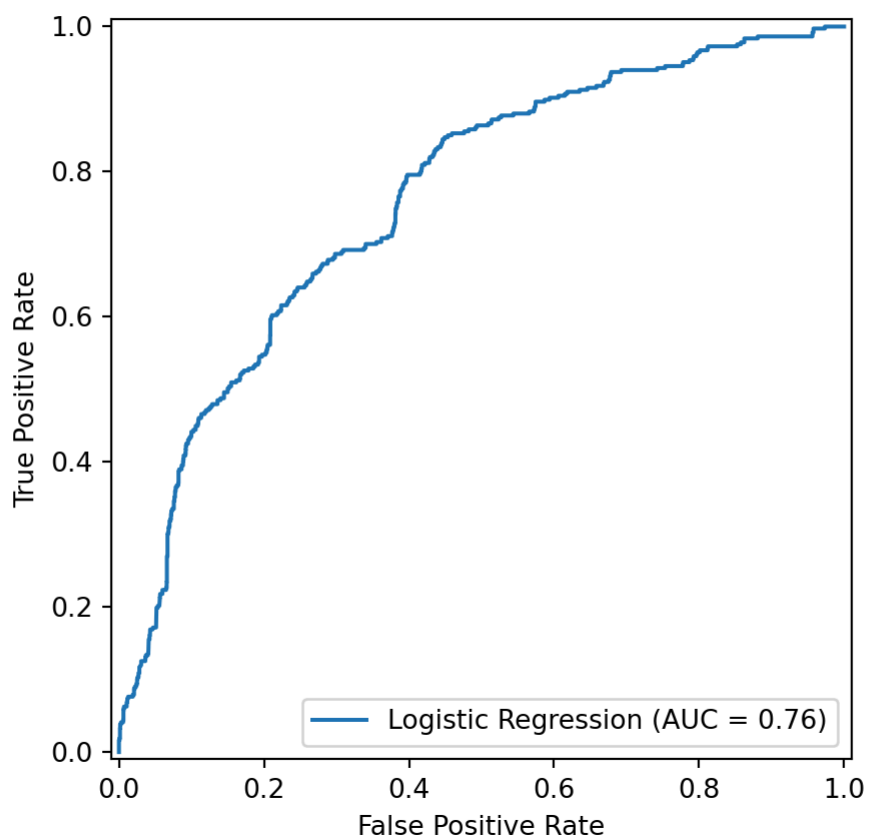
Scoring visualizations - confusion matrix

```
from sklearn.metrics import ConfusionMatrixDisplay
cm = confusion_matrix(y, m.predict(X), labels=m.classes_)
disp = ConfusionMatrixDisplay(cm).plot()
plt.show()
```



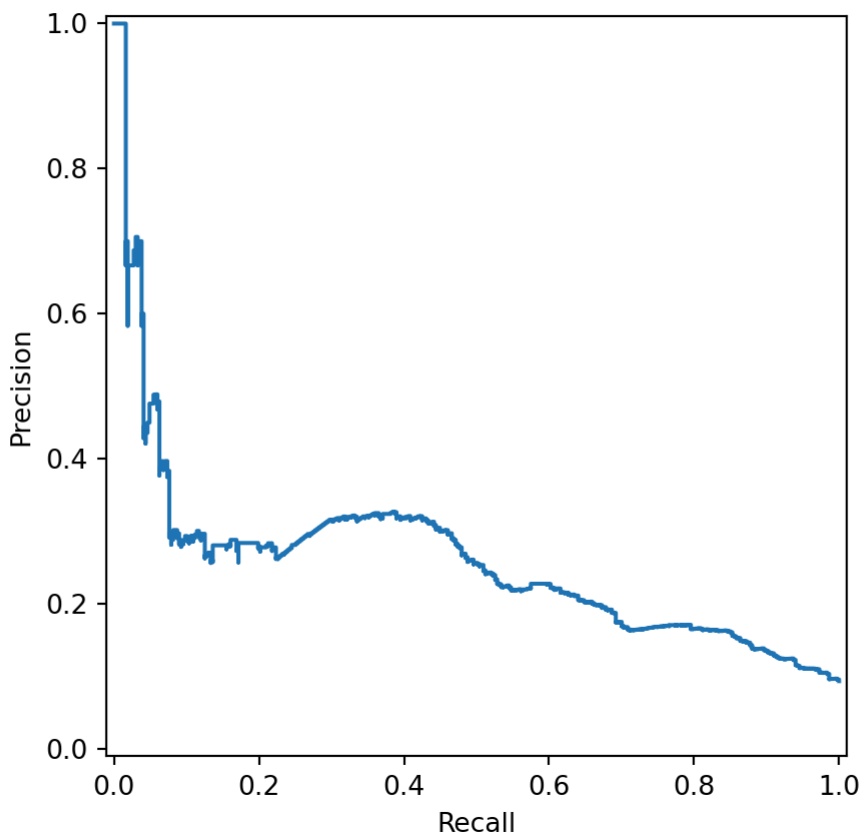
Scoring visualizations - ROC curve

```
from sklearn.metrics import auc, roc_curve, RocCurveDisplay
fpr, tpr, thresholds = roc_curve(y, m.predict_proba(X)[:,1])
roc_auc = auc(fpr, tpr)
disp = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                       estimator_name='Logistic Regression').plot()
plt.show()
```



Scoring visualizations - Precision Recall

```
from sklearn.metrics import precision_recall_curve, PrecisionRecallDisplay
precision, recall, _ = precision_recall_curve(y, m.predict_proba(X)[:,1])
disp = PrecisionRecallDisplay(precision=precision, recall=recall).plot()
plt.show()
```



StratifiedKFold

For classification problems, `StratifiedKFold` is preferred over `KFold` - it ensures each fold preserves the same class proportions as the full dataset, which is particularly important for imbalanced classes.

```
from sklearn.model_selection import StratifiedKFold
print(f'overall spam%={y.mean():.3f}')
```

```
overall spam%=0.094
```

KFold

```
cv = KFold(5, shuffle=True, random_state=12
for i, (train, test) in enumerate(cv.split(
  print(f'fold {i+1}: spam%={y.iloc[test].me
```

```
fold 1: spam%=0.078
fold 2: spam%=0.107
fold 3: spam%=0.097
fold 4: spam%=0.085
fold 5: spam%=0.101
```

StratifiedKFold

```
cv = StratifiedKFold(5, shuffle=True, rando
for i, (train, test) in enumerate(cv.split(
  print(f'fold {i+1}: spam%={y.iloc[test].me
```

```
fold 1: spam%=0.094
fold 2: spam%=0.093
fold 3: spam%=0.093
fold 4: spam%=0.093
fold 5: spam%=0.094
```

MNIST

MNIST handwritten digits

```python
from sklearn.datasets import load_digits
digits = load_digits(as_frame=True)
```

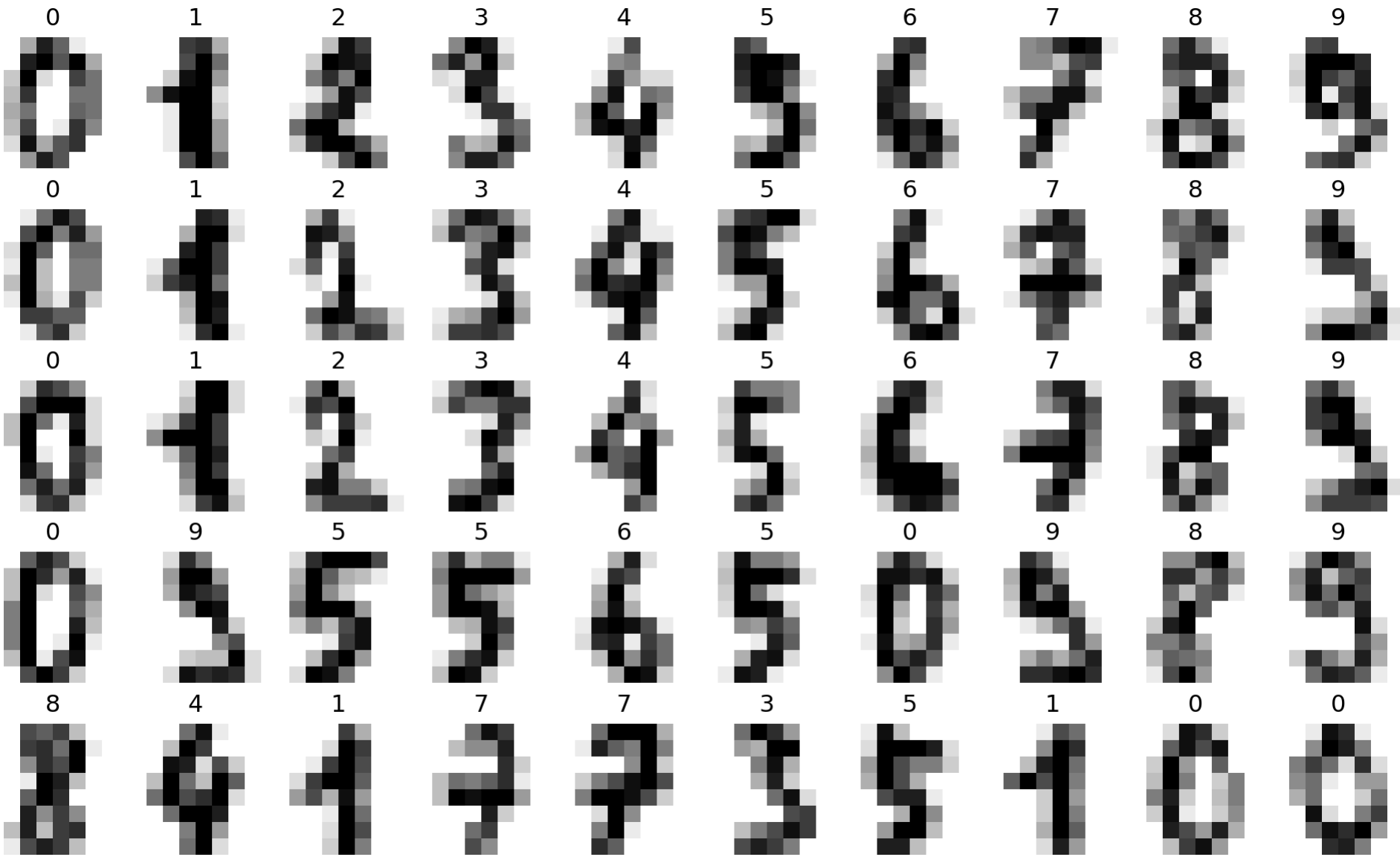
```python
X = digits.data
X
```

```
      pixel_0_0  pixel_0_1  pixel_0_2  ...  pixel_7_5  pixel_7_6  p
0           0.0        0.0        5.0  ...        0.0        0.0
1           0.0        0.0        0.0  ...       10.0        0.0
2           0.0        0.0        0.0  ...       16.0        9.0
3           0.0        0.0        7.0  ...        9.0        0.0
4           0.0        0.0        0.0  ...        4.0        0.0
...         ...        ...        ...  ...        ...        ...
1792        0.0        0.0        4.0  ...        9.0        0.0
1793        0.0        0.0        6.0  ...        6.0        0.0
1794        0.0        0.0        1.0  ...        6.0        0.0
1795        0.0        0.0        2.0  ...       12.0        0.0
1796        0.0        0.0       10.0  ...       12.0        1.0

[1797 rows x 64 columns]
```

```python
y = digits.target
y
```

```
0       0
1       1
2       2
3       3
4       4
       ..
1792    9
1793    0
1794    8
1795    9
1796    8
Name: target, Length: 1797, 
```

Example digits



Doing things properly - train/test split

To properly assess our modeling we will create a training and testing set of these data, only the training data will be used to learn model coefficients or hyperparameters, test data will only be used for final model scoring.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, shuffle=True, stratify=y, random_state=1234
)
```

Multiclass logistic regression

Fitting a multiclass logistic regression model in recent versions of sklearn uses multinomial logistic regression by default. We can use `GridSearchCV` with `StratifiedKFold` to explore the available solver options (all of which support `penalty=None`).

```
mc_log_cv = GridSearchCV(
  LogisticRegression(penalty=None, max_iter = 5000),
  param_grid = {"solver": ["lbfgs", "newton-cg", "sag", "saga"]},
  cv = StratifiedKFold(10, shuffle=True, random_state=12345)
).fit(
  X_train, y_train
)
```

```
mc_log_cv.best_estimator_
```

```
LogisticRegression(max_iter=5000, penalty=None, solver='saga')
```

```
mc_log_cv.best_score_
```

```
np.float64(0.9625550964187328)
```

```
for param, score in zip(mc_log_cv.cv_results_["params"], mc_log_cv.cv_results_["mean_test_sco
  print(f"{param=}, {score=}")
```

```
param={'solver': 'lbfgs'}, score=np.float64(0.9542699724517906)
param={'solver': 'newton-cg'}, score=np.float64(0.9592493112947 66)
```

Model coefficients

```python
pd.DataFrame(
  mc_log_cv.best_estimator_.coef_
)
```

```
    0         1         2         3   ...        60        61        62        63
0  0.0 -0.004279 -0.038662  0.030116  ... -0.036485 -0.056842 -0.047864 -0.015896
1  0.0 -0.000220 -0.150098  0.145915  ...  0.071136  0.194559  0.227603  0.142881
2  0.0  0.038770  0.039576  0.047945  ...  0.086936  0.210536  0.472231  0.173976
3  0.0  0.046396 -0.161494  0.064254  ...  0.117194  0.083336 -0.045956 -0.116159
4  0.0 -0.004034  0.023236 -0.313809  ... -0.184223 -0.234043 -0.096115 -0.003016
5  0.0  0.070263  0.472733  0.068087  ... -0.069761 -0.083259 -0.046395 -0.017624
6  0.0 -0.008248 -0.219789 -0.076978  ... -0.022557  0.098022  0.065408 -0.054713
7  0.0  0.024608  0.046442  0.079125  ... -0.235789 -0.349953 -0.137181 -0.010544
8  0.0 -0.001529  0.066322 -0.271446  ...  0.245932 -0.001169 -0.168421 -0.120803
9  0.0 -0.161727 -0.078267  0.226790  ...  0.027618  0.138812 -0.223311  0.021897

[10 rows x 64 columns]
```

```python
mc_log_cv.best_estimator_.coef_.shape
```

```
(10, 64)
```

```python
mc_log_cv.best_estimator_.intercept_
```

```
array([ 0.00589, -0.06445,  0.00283,  0.01217,  0.04331,  0.00833, -0.00055,  0.00992,
        0.0295 , -0.04696])
```

Confusion Matrix

Within sample

```python
accuracy_score(
  y_train,
  mc_log_cv.best_estimator_.predict(X_train)
)
```

```
1.0
```

```python
confusion_matrix(
  y_train,
  mc_log_cv.best_estimator_.predict(X_train)
)
```

```
array([[119,   0,   0,   0,   0,   0,   0,   0,   0,   0],
       [  0, 122,   0,   0,   0,   0,   0,   0,   0,   0],
       [  0,   0, 118,   0,   0,   0,   0,   0,   0,   0],
       [  0,   0,   0, 123,   0,   0,   0,   0,   0,   0],
       [  0,   0,   0,   0, 121,   0,   0,   0,   0,   0],
       [  0,   0,   0,   0,   0, 122,   0,   0,   0,   0],
       [  0,   0,   0,   0,   0,   0, 121,   0,   0,   0],
       [  0,   0,   0,   0,   0,   0,   0, 120,   0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0, 116,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0, 121]])
```

Out of sample

```python
accuracy_score(
  y_test,
  mc_log_cv.best_estimator_.predict(X_test)
)
```

```
0.968013468013468
```

```python
confusion_matrix(
  y_test,
  mc_log_cv.best_estimator_.predict(X_test),
  labels = digits.target_names
)
```

```
array([[59,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 58,  0,  1,  0,  0,  0,  0,  0,  1],
       [ 0,  0, 59,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  1, 56,  0,  2,  0,  0,  1,  0],
       [ 0,  1,  0,  0, 58,  0,  0,  0,  1,  0],
       [ 0,  0,  0,  0,  0, 56,  0,  0,  1,  3],
       [ 0,  0,  0,  0,  0,  0, 60,  0,  0,  0],
       [ 0,  0,  0,  0,  2,  0,  0, 56,  0,  1],
       [ 0,  1,  1,  0,  0,  0,  0,  0, 56,  0],
       [ 0,  0,  0,  0,  0,  1,  0,  0,  1, 57]])
```

Report

```python
print( classification_report(
  y_test,
  mc_log_cv.best_estimator_.predict(X_test)
) )
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        59
           1       0.97      0.97      0.97        60
           2       0.97      1.00      0.98        59
           3       0.98      0.93      0.96        60
           4       0.97      0.97      0.97        60
           5       0.95      0.93      0.94        60
           6       1.00      1.00      1.00        60
           7       1.00      0.95      0.97        59
           8       0.93      0.97      0.95        58
           9       0.92      0.97      0.94        59

    accuracy                           0.97       594
   macro avg       0.97      0.97      0.97       594
weighted avg       0.97      0.97      0.97       594
```

Prediction

```python
mc_log_cv.best_estimator_.predict(X_test)
```

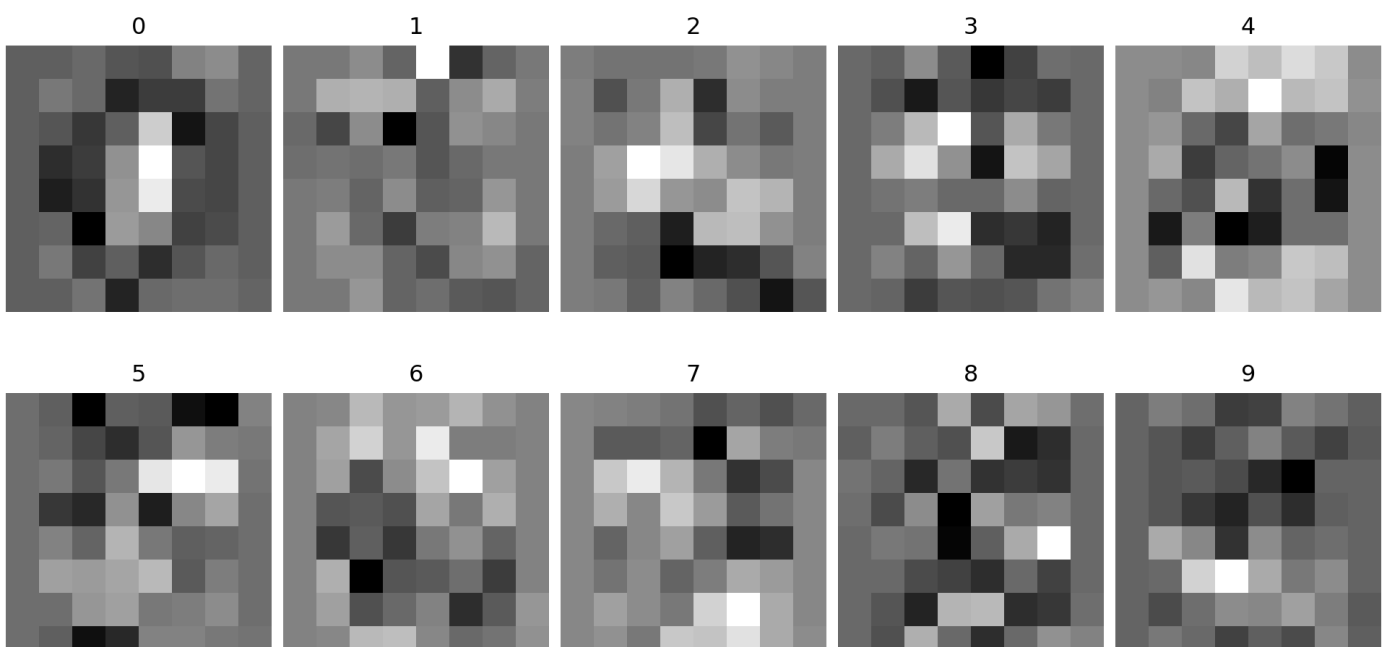
```
array([1, 6, 1, 9, 6, 7, 8, 7, 2, 4, 0, 7, 1, 7, 6, 6, 0,
       5, 0, 4, 3, 2, 3, 1, 7, 0, 8, 2, 2, 5, 1, 2, 7, 3,
       1, 2, 6, 2, 5, 8, 1, 5, 6, 7, 1, 9, 6, 7, 9, 4, 9,
       9, 6, 3, 5, 4, 1, 3, 8, 6, 0, 1, 6, 4, 8, 1, 2, 8,
       0, 3, 8, 3, 0, 0, 6, 1, 9, 1, 7, 0, 3, 1, 1, 6, 1,
       1, 9, 1, 4, 3, 9, 9, 8, 3, 3, 4, 4, 7, 5, 3, 9, 9,
       8, 1, 9, 6, 2, 8, 5, 8, 6, 4, 4, 1, 4, 1, 5, 0, 6,
       9, 2, 5, 5, 2, 8, 8, 5, 9, 0, 0, 8, 6, 4, 1, 3, 3,
       8, 9, 4, 4, 4, 4, 4, 0, 2, 0, 3, 4, 5, 2, 2, 8, 8,
       3, 7, 7, 1, 7, 2, 4, 9, 9, 9, 4, 2, 2, 9, 0, 0, 7,
       6, 5, 7, 3, 4, 2, 5, 5, 2, 0, 5, 9, 6, 2, 9, 1, 8,
       1, 2, 2, 5, 7, 4, 6, 4, 9, 9, 8, 5, 6, 7, 9, 4, 2,
       6, 1, 4, 7, 3, 2, 5, 6, 5, 5, 6, 0, 3, 8, 1, 1, 9,
       2, 7, 8, 0, 4, 0, 7, 0, 5, 2, 7, 6, 3, 4, 9, 5, 7,
       3, 8, 6, 8, 1, 5, 2, 8, 2, 8, 6, 6, 8, 2, 2, 7, 2,
       3, 6, 8, 9, 4, 2, 3, 8, 7, 3, 5, 4, 8, 3, 9, 6, 3,
       2, 0, 9, 7, 5, 0, 3, 2, 6, 6, 6, 5, 8, 8, 6, 7, 0,
       1, 3, 2, 5, 9, 7, 8, 8, 4, 6, 3, 4, 6, 1, 4, 2, 3,
       9, 2, 0, 2, 9, 8, 3, 6, 3, 9, 7, 8, 0, 2, 2, 0, 1,
       7, 2, 1, 2, 7, 6, 3, 9, 2, 4, 1, 2, 8, 7, 0, 4, 7,
       2, 4, 4, 8, 4, 4, 1, 8, 0, 7, 6, 9, 9, 0, 5, 7, 0,
       9, 9, 4, 5, 1, 5, 2, 3, 2, 1, 5, 9, 9, 8, 7, 9, 1,
       3, 6, 6, 9, 8, 8, 7, 7, 5, 3, 5, 7, 4, 8, 0, 6, 5,
       2, 0, 1, 2, 5, 8, 7, 8, 5, 7, 4, 3, 6, 2, 7, 8, 0,
       9, 9, 4, 3, 4, 5, 3, 4, 6, 0, 5, 0, 6, 2, 3, 0, 7,
       2, 4, 3, 0, 3, 0, 0, 4, 8, 7, 4, 6, 1, 8, 6, 3, 4,
       9, 6, 2, 6, 4, 5, 5, 8, 5, 5, 4, 1, 5, 8, 5, 7, 1,
       7, 5, 7, 3, 0, 9, 9, 7, 1, 0, 0, 5, 6, 5, 0, 6, 0,
       0, 3, 1, 1, 9, 9, 5, 3, 3, 2, 9, 4, 7, 5, 0, 0, 1,
```

```python
mc_log_cv.best_estimator_.predict_proba(X_test)
```

```
array([[0.     , 0.99364, 0.     , 0.     , 0.00631,
        0.     , 0.     , 0.     , 0.00005, 0.     ],
       [0.     , 0.     , 0.     , 0.     , 0.     ,
        0.     , 0.99998, 0.     , 0.00002, 0.     ],
       [0.     , 0.99996, 0.     , 0.     , 0.     ,
        0.     , 0.00001, 0.     , 0.00003, 0.     ],
       [0.     , 0.     , 0.     , 0.00001, 0.     ,
        0.     , 0.     , 0.     , 0.     , 0.99999],
       [0.     , 0.     , 0.     , 0.     , 0.     ,
        0.     , 1.     , 0.     , 0.     , 0.     ],
       [0.     , 0.     , 0.     , 0.00051, 0.     ,
        0.     , 0.     , 0.99946, 0.00003, 0.     ],
       [0.     , 0.     , 0.     , 0.     , 0.     ,
        0.     , 0.     , 0.     , 1.     , 0.     ],
       [0.     , 0.     , 0.     , 0.     , 0.     ,
        0.     , 0.     , 1.     , 0.     , 0.     ],
       [0.     , 0.     , 0.99998, 0.00002, 0.     ,
        0.     , 0.     , 0.     , 0.     , 0.     ],
       [0.     , 0.     , 0.     , 0.     , 1.     ,
        0.     , 0.     , 0.     , 0.     , 0.     ],
       [1.     , 0.     , 0.     , 0.     , 0.     ,
        0.     , 0.     , 0.     , 0.     , 0.     ],
       [0.     , 0.     , 0.     , 0.     , 0.     ,
        0.     , 0.     , 1.     , 0.     , 0.     ],
       [0.     , 1.     , 0.     , 0.     , 0.     ,
        0.     , 0.     , 0.     , 0.     , 0.     ],
       [0.     , 0.     , 0.     , 0.     , 0.00003,
        0.     , 0.     , 0.99996, 0.     , 0.     ],
       [0.     , 0.00002, 0.     , 0.     , 0.     ,
```

Examining the coefs

```
coef_img = mc_log_cv.best_estimator_.coef_.reshape(10,8,8)

fig, axes = plt.subplots(nrows=2, ncols=5, figsize=(10, 5), layout="constrained")
axes2 = [ax for row in axes for ax in row]

for ax, image, label in zip(axes2, coef_img, range(10)):
    ax.set_axis_off()
    img = ax.imshow(image, cmap=plt.cm.gray_r, interpolation="nearest")
    txt = ax.set_title(f"{label}")

plt.show()
```



Example 1 - DecisionTreeClassifier

Using these data we will now fit a `DecisionTreeClassifier`, we will employ `GridSearchCV` to tune some of the parameters (`max_depth` at a minimum) - see the full list here.

```
from sklearn.datasets import load_digits
digits = load_digits(as_frame=True)


X, y = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, shuffle=True, stratify=y, random_state=1234
)
```

Example 2 - GridSearchCV w/ Multiple models (Trees vs Forests)